Hand Proofs for Compositional Rounding Certificates

Anonymous

Status. These are mathematical proofs for the semantics and constructions implemented in the accompanying artifact. They are not a proof-assistant development. Executable tests instantiate and mutate the constructions, but finite testing is not substituted for the general arguments below.

1 Finite semantics

Fix a signed word width $w \geq 2$, a power-of-two scale $q < 2^{w-1}$, and

$$L = -2^{w-1}, \qquad H = 2^{w-1} - 1, \qquad \mathcal{W} = [L, H] \cap \mathbb{Z}.$$

For $n = qa + b$ with $0 \leq b < q$, define

$$\begin{aligned} \mathrm{R}_q^{\mathrm{floor}}(n) &= a, \\ \mathrm{R}_q^{\mathrm{rne}}(n) &= a + \mathbf{1}\{2b > q \text{ or } (2b = q \text{ and } a \text{ odd})\}. \end{aligned}$$

The selected unsaturated rounder is R_q. Define

$$\mathrm{Q}(n) = \mathrm{clip}(\mathrm{R}_q(n)), \qquad \mathrm{clip}(z) = \min(H, \max(L, z)).$$

All sums and products inside an argument to Q are exact integers; there is no wrapping arithmetic.

An instance has dimension d, horizon N, initial state $h_0 \in \mathcal{W}^d$, matrices $A_t \in \mathcal{W}^{d \times d}$, and input boxes $U_t = \prod_i [\ell_{ti}, u_{ti}] \cap \mathbb{Z}^d$. Its sequential recurrence is

$$h_t[i] = \mathrm{Q}\left(\sum_k A_t[i,k] h_{t-1}[k] + q b_t[i] \right).$$

A stored affine pair (A, b) acts by the same formula. If a left pair (A, b) is followed by a right pair (C, e), the lowering stores

$$(A, b) \star (C, e) = (P, z), \quad P_{ij} = \mathrm{Q}\left(\sum_k C_{ik} A_{kj} \right), \quad z_i = \mathrm{Q}\left(\sum_k C_{ik} b_k + q e_i \right).$$

Each declared prefix association tree applies $\star$ at its internal nodes. Every causal prefix in one chunk is applied to the same incoming lowered carry; only the last prefix result becomes the next chunk carry.

For tolerances $\tau_{ti} \geq 0$, safety means

$$\forall b_1 \in U_1, \ldots, b_N \in U_N : \quad |\widehat{h}_t[i] - h_t[i]| \leq \tau_{ti} \quad \text{for all } t, i.$$

A counterexample is ordered first by output time and then lexicographically by the flattened time-major input prefix.

2 Rounding identities

Define the pre-saturation residual

$$\rho_q(n) = q\,\mathrm{R}_q(n) - n.$$

Lemma 2.1 (Sound translation periods)**.** *Let $M = q$ for floor rounding and $M = 2q$ for round-to-nearest-even. For every $n, k \in \mathbb{Z}$,*

$$\mathrm{R}_q(n + Mk) = \mathrm{R}_q(n) + (M/q)k, \qquad \rho_q(n + Mk) = \rho_q(n).$$

For nearest-even rounding, period q is invalid in general.

Proof. For floor rounding, adding qk changes the Euclidean quotient by k and preserves the remainder. For nearest-even, adding $2qk$ changes the quotient by the even integer $2k$ and preserves the remainder. Consequently, both the comparison of $2b$ with q and the parity tie-break are unchanged. Substitution in the definition of ρ_q proves residual equality. For the final statement, take $q = 2$, $n = 1$, and $k = 1$: $\mathrm{R}_2^{\mathrm{rne}}(1) = 0$ but $\mathrm{R}_2^{\mathrm{rne}}(3) = 2$, not 1. □

Lemma 2.2 (Clipping is monotone and nonexpansive)**.** *For integers $x \le y$, $\mathrm{clip}(x) \le \mathrm{clip}(y)$ and $0 \le \mathrm{clip}(y) - \mathrm{clip}(x) \le y - x$.*

Proof. The map $z \mapsto \min(H, \max(L, z))$ is the composition of two monotone, one-Lipschitz maps on the ordered integers. □

Lemma 2.3 (Exact integer rounding modulus)**.** *For $D \in \mathbb{Z}_{\ge 0}$ define*

$$\omega_q(D) = \max_{n \in \mathbb{Z}} |\,\mathrm{R}_q(n + D) - \mathrm{R}_q(n)|.$$

For floor rounding,

$$\omega_q(D) = \lceil D/q \rceil\,.$$

For nearest-even, $\omega_1(D) = D$. If q is even and $D = kq + r$ with $0 \le r < q$, then

$$\omega_q(D) = k + \mathbf{1}\{r > 0\} + \mathbf{1}\{r = 0 \text{ and } k \text{ odd}\}.$$

The same expressions upper-bound the difference after clipping.

Proof. Monotonicity lets us omit the absolute value. For floor rounding, crossing an interval of integer length D crosses at most $\lceil D/q \rceil$ multiples of q, and a starting residue immediately before the largest possible number of boundaries attains the bound.

For nearest-even with $q = 1$, all integer arguments are exact. Let even q be fixed and put

$$J(n) = \left\lfloor \frac{n + q/2}{q} \right\rfloor, \qquad \chi(n) = \mathbf{1}\{n \equiv q/2 \pmod{2q}\}.$$

Nearest-even satisfies $\mathrm{R}_q(n) = J(n) - \chi(n)$: J rounds half-integers upward, and χ subtracts one exactly when the lower quotient is even. The increment of J over distance D is at most $\lceil D/q \rceil$. If $\chi(n) = 0$, the correction at the endpoint cannot increase that increment. If $\chi(n) = 1$ and $r > 0$, then the half-up increment from $n = q/2$ is k while $\chi(n + D) = 0$, yielding $k + 1$. If $\chi(n) = 1$ and $r = 0$, then the half-up increment is k and the endpoint is another corrected tie precisely when k is even; the result is $k + \mathbf{1}\{k \text{ odd}\}$. Choosing $n = q/2$ attains each stated maximum. Lemma 2.2 proves the clipped upper bound. □

3 Paired symbolic enclosure

The producer associates each value with (Z, E, I): an integer affine expression Z, a rational affine expression E, and an integer interval I. Expressions range over named input, quotient, residual, and clipped-value atoms. Each atom has an interval. The key invariant is existential in auxiliary atoms but universal in concrete inputs.

Definition 3.1 (Paired invariant)**.** *For every concrete input in the current subbox, there exists one valuation of all currently named atoms within their recorded intervals such that Z, E, and the concrete value agree, and that value belongs to I. One atom name has one value throughout the context; the Cartesian product of atom intervals may contain valuations that no concrete input realizes.*

Integer affine addition and scaling update both expressions algebraically and propagate I. The implementation intersects three independently sound ranges when available: direct interval propagation, the range of Z, and the integral points in the range of E.

For a numerator symbolic value, let z be its exact integer expression and let n be its concrete value under an invariant valuation. The producer creates or reuses a quotient atom for exactly identical affine-expression keys. It also creates a residual atom under one of three policies:

1. *fresh*: a distinct residual atom at every quantization;
2. *syntactic*: reuse only for an identical numerator-expression key;
3. *residue*: reuse when every affine coefficient and the constant agree modulo M, where M is from Lemma 2.1.

A residual interval is computed from a sound overapproximation of all possible numerator residues modulo M. The rational output expression is

$$E' = E/q + \eta/q,$$

where η is the residual atom. The exact integer expression is either the constant rounded value or a quotient atom. Saturation then keeps these forms if clipping is inactive, replaces them by a constant when all values clip to one endpoint, or introduces a clipped-value atom otherwise.

Lemma 3.2 (Residue-set containment)**.** *The modular sumset procedure in* `possible_residues` *contains the residue modulo M of every valuation of an integer affine expression within the recorded atom intervals.*

Proof. Start from the constant residue. For a term ca, the sequence $c\ell, c(\ell+1), \ldots$ modulo M has period $M/\gcd(|c|, M)$ when $c \neq 0$ and period one otherwise. Enumerating the smaller of the interval cardinality and that period therefore includes every possible term residue. Taking modular sumsets preserves containment after each term. Discarding cross-atom correlations can add residues but cannot remove the concrete one. □

Theorem 3.3 (Paired-enclosure soundness)**.** *Every symbolic operation in the implementation preserves Definition 3.1; this includes all three residual policies and saturation. Consequently, every returned paired bound contains the concrete lowered-minus-reference difference for every input in the supplied subbox.*

Proof. Constants and input atoms establish the invariant directly. Affine addition and integer scaling preserve equality of both expressions with the concrete value. Every propagated interval contains the corresponding operation on the concrete operands; intersections remain sound because each intersectand contains the same value.

Consider quantization. The integer-expression range contains n, so monotonicity of R_q makes the stored quotient interval contain $\mathrm{R}_q(n)$. An exact-expression key denotes the same affine function under the same atom valuation; hence reuse of a quotient atom assigns the same concrete quotient at every occurrence.

For residuals, a fresh atom can be assigned $\rho_q(n)$. Under syntactic sharing, identical expressions have identical values and residuals. Under residue sharing, congruent coefficient maps give congruent values modulo M for every integer atom valuation, so Lemma 2.1 gives equal residuals. Lemma 3.2 ensures that the stored residual interval contains the assigned residual. Since the pre-quantization rational expression equals n,

$$E' = n/q + \rho_q(n)/q = \mathrm{R}_q(n).$$

The quotient atom gives the same integer. Thus the unsaturated result satisfies the invariant.

If its interval lies wholly below L or above H, clipping is the corresponding constant. If clipping is inactive, nothing changes. In the partial case, assign the new clipped-value atom the concrete clipped quotient; its interval contains that value. Reuse is keyed by the exact pre-clipped quotient expression, so reused occurrences denote the same integer function and therefore the same clipped value. This proves preservation through saturation.

Matrix-summary entries are concrete quantizations of constants. Bias summaries, tree composition, reference updates, and applications to chunk carries are compositions of the proven operations. At an output, the Z difference, E difference, and direct interval difference all contain the concrete lowered-minus-reference value, so their intersection does as well. □

Remark 3.4. *The theorem is a soundness theorem, not a polynomial-time guarantee in an unbounded binary-encoded scale. A residue set can contain M elements. The artifact instances use power-of-two scales no larger than eight in the frozen suite, and the reductions use $q = 2$.*

4 Exact leaves and ordered covers

A cover node is either a safe leaf, a singleton witness, or a split. The checker requires each split to use the first nonsingleton coordinate of the flattened prefix box. Its left child covers the lower interval and is checked first; a right child is forbidden if the left subtree already contains a witness.

Lemma 4.1 (Exact scalar dynamic program)**.** *For a scalar instance and any scalar prefix box admitted by the checker, the dynamic program in* `exact_box.py` *computes the exact minimum and maximum output difference for that box, for left, right, or balanced declared association.*

Proof. After step t, define a state triple (r, c, u) consisting of the sequential reference state r, the incoming lowered carry c of the current chunk, and the exact tuple u of inputs already chosen in that chunk. Inductively, the dynamic-program state set is exactly the set of reachable triples for all prefixes in the supplied box. The base triple contains the common initial state and an empty local tuple. For every reachable triple, the transition enumerates every legal next input code, applies one exact sequential update, appends the code to u, and recomputes the declared prefix summary with the declared association tree. Thus it adds exactly every reachable successor and no unreachable successor. At a chunk boundary it replaces the incoming carry by the last lowered output and clears u, matching the semantics. At the requested output, the set of recorded differences is therefore exactly the image of the supplied box. Its minimum and maximum are exact. □

Proposition 4.2 (Exact-leaf evaluator)**.** *For every box on which* `exact_box_range` *returns rather than raising* `ExactBoxLimit`*, its returned component intervals are exact ranges.*

Proof. Small boxes are enumerated completely. Large admitted scalar boxes use Lemma 4.1. Large nonscalar boxes outside the enumeration bound are rejected rather than approximated. □

Lemma 4.3 (Ordered-cover integrity)**.** *Let a checked tree cover a prefix box B.*

1. *If checking returns* safe*, every point in B satisfies the selected output tolerance.*
2. *If checking returns witness x, then x is the lexicographically least violating point in B.*

Proof. Proceed by structural induction. At a safe leaf, Proposition 4.2 supplies the exact range, the submitted bounds contain it, and the bounds lie within the tolerance. At a witness leaf, the box is a singleton and replay establishes a violation, so that point is the first and only member.

At a split, the required coordinate and cut partition B into a lower left box and an upper right box; every left point precedes every right point in lexicographic order because all earlier coordinates are fixed by the first-nonsingleton rule. If the left subtree yields a witness, the checker forbids a right subtree and induction makes that witness first in B. If the left subtree is safe, any violation lies in the right subtree; induction then gives the first such point. Two safe children prove their union safe. □

Theorem 4.4 (Checker contract)**.** *If the checker accepts a safe certificate, the instance satisfies every-output safety. If it accepts a violation certificate, its reported output is the earliest failing output and its witness is the lexicographically first violating input prefix at that output.*

Proof. The checker requires consecutive roots starting at the first output. Apply Lemma 4.3 to each. In a safe certificate, every horizon root is safe. In a violation certificate, all preceding roots are safe and the final root yields its first witness. The embedded instance, root count, earliest-output field, and top-level witness are recomputed or cross-checked rather than trusted. □

Theorem 4.5 (Finite ordered completeness without resource cutoffs)**.** *Assume the leaf evaluator admits every singleton and the producer's abstract bounds are sound. If node and time limits are removed, canonical bisection produces a conclusive safe or violation certificate for every finite instance. A violation result uses the ordering of Theorem 4.4.*

Proof. Each non-singleton split strictly reduces the cardinality of both child boxes, so recursive subdivision of a finite integer box terminates. A sound abstract proof may stop earlier. Otherwise every leaf eventually becomes singleton, where exact replay decides it. Outputs are processed in increasing time and subboxes in left-before-right lexicographic order, so the first encountered witness has the required ordering. A safe instance exhausts all roots with safe leaves. □

5 Temporal composition

For a nonnegative integer difference D, let $\omega_q(D)$ be Lemma 2.3. For a stored matrix A define the monotone component map

$$T_A(v)_i = \omega_q\left(\sum_k |A_{ik}| v_k\right).$$

Repeated composition along the sequential steps of a chunk prefix is $\Phi_{j,p}$.

Let $F_{j,p}(h,u)$ be the sequential execution of prefix p in chunk j from carry h, and $G_{j,p}(h,u)$ the lowered prefix. Suppose a local same-carry result establishes

$$|G_{j,p}(h,u) - F_{j,p}(h,u)| \le \varepsilon_{j,p} \quad (h \in C_j,\ u \in U_j),$$

and C_j contains the actual incoming carries of both executions.

Theorem 5.1 (Carry-aware temporal error contract)**.** *If the incoming carry difference is bounded by E_{j-1}, then*

$$|\widehat{h}_{j,p} - h_{j,p}| \le E_{j,p} := \min\{(H-L)\mathbf{1},\ \varepsilon_{j,p} + \Phi_{j,p}(E_{j-1})\}$$

componentwise. The last prefix bound is a valid incoming bound for the next chunk; equal initial states give $E_0 = 0$.

Proof. For actual incoming carries $\widehat{c}, c$ and common local input u, insert a sequential execution from $\widehat{c}$:

$$\begin{aligned}|G_{j,p}(\widehat{c},u) - F_{j,p}(c,u)| \le\ & |G_{j,p}(\widehat{c},u) - F_{j,p}(\widehat{c},u)| \\ & + |F_{j,p}(\widehat{c},u) - F_{j,p}(c,u)|.\end{aligned}$$

The first term is bounded by the local contract because $\widehat{c} \in C_j$. At one sequential step, the exact numerator difference in coordinate i is at most $\sum_k |A_{ik}| v_k$; Lemmas 2.3 and 2.2 bound the rounded-and-clipped difference by $T_A(v)_i$. Iterating yields $\Phi_{j,p}(E_{j-1})$. Both outputs lie in $\mathcal{W}^d$, giving the word-span bound. The lowered semantics passes only the final prefix result to the next chunk, so induction over chunks completes the proof. □

6 Independent state blocks

Let $B_1, \dots, B_s$ partition the state coordinates. Assume every A_t is block diagonal for this same partition and the input domain is the stated Cartesian product.

Lemma 6.1 (Factorized executions)**.** *The reference state and every lowered prefix are the direct product of their projected block executions.*

Proof. A sequential row in one block reads no state coordinate outside that block. The product of two matrices with the common block-diagonal pattern has zero cross-block entries; $\mathrm{Q}(0) = 0$, so quantized matrix composition preserves the pattern. Bias composition and application to a carry likewise stay within each block. Induction over each association tree and then over chunk boundaries proves the result. □

Theorem 6.2 (Block safety and ordered witness lifting)**.** *The full instance is safe exactly when every projected block instance is safe. If some block fails, the earliest full-instance failing output is the minimum of the block failure times. At that time, lift each first block witness by filling every other coordinate with its input-box minimum. The lexicographically smallest such lift is the first full-instance violating prefix.*

Proof. By Lemma 6.1, a componentwise product contract holds exactly when it holds in every factor, proving safety and earliest time. A lifted block witness is legal and remains violating. Consider any full violation at the minimum time and a block it violates. Its projection cannot precede that block's first witness. Filling all other coordinates with minima gives a vector no later than the original full point: the first difference is either an outside-block minimum or the earlier block-witness coordinate. Taking the least lifted candidate over failing blocks proves global minimality. □

7 Rounding-only complexity barrier

We state decision problems under the explicit finite semantics above. An instance is encoded by its dimensions, horizon, word width, scale, matrices, boxes, trees, and tolerances. A violating input prefix is polynomial in that encoding and can be replayed in polynomial time using integer arithmetic of polynomial bit length. Thus unsafety is in NP and safety is in coNP for both restricted families below.

Lemma 7.1 (Two-step Boolean annihilator)**.** *At scale $q = 2$, let $x \in \{0, 1\}$ and scratch $s = 0$. First preserve x and set $s = \mathrm{R}_2(x)$. Then set $x' = \mathrm{R}_2(2x - 4s)$. Under floor or nearest-even, sequential execution yields $x' = x$. Quantized composition of the two stored matrices has an all-zero output row for x, so the lowered two-step prefix yields zero. Every value fits a signed three-bit word and clipping is inactive.*

Proof. For $x \in \{0, 1\}$, both selected rounders give $s = 0$, so sequentially $x' = \mathrm{R}_2(2x) = x$. In the stored matrix product, the output coefficient of the original x is $\mathrm{Q}(2 \cdot 2 + (-4) \cdot 1) = \mathrm{Q}(0) = 0$ and every other coefficient in that row is zero. Applying the summary therefore gives zero. Coefficients belong to $[-4, 3]$ and observed states belong to $\{0, 1\}$. □

Theorem 7.2 (Fixed-width all-output safety is coNP-complete)**.** *Universal bit-exact every-output equivalence is coNP-complete even with scale two, signed three-bit words, two-step chunks, floor or nearest-even rounding, and no active clipping, when dimension and horizon may grow.*

Proof. Membership was noted above. Reduce circuit UNSAT for an AND/NOT Boolean circuit. Allocate one state coordinate for each input and gate plus one scratch coordinate. A load step writes Boolean input biases, followed by an identity step. For NOT, replace a target by $1 - x$ using coefficient -2 and bias one. For AND, use $\mathrm{R}_2(x + y)$: it is one exactly when $x = y = 1$; a repeated input is copied with coefficient two. Follow every gate update by an identity step. Therefore every ordinary two-step summary agrees with sequential execution, and all registers remain Boolean.

Apply Lemma 7.1 to the circuit-output register in the final chunk. The two executions agree at all earlier outputs; at the last output, the lowered value is zero and the reference value is the circuit output. With zero tolerances, the instance is safe exactly when the circuit is false on every Boolean assignment. The construction is polynomial and does not activate clipping. □

Theorem 7.3 (Five-state final-output safety is coNP-complete)**.** *With state dimension fixed at five, scale two, floor rounding, two-step chunks, and only one nonvacuous final-output tolerance, universal safety is coNP-complete when word width and horizon may grow.*

Proof. Membership again follows from a replayable input code. Reduce CNF UNSAT with variables packed into one nonnegative code $X \in [0, 2^m - 1]$. Choose width $\max(3, m + 1)$. The five registers are packed assignment X, extraction register E, clause accumulator C, formula accumulator P, and scratch F.

To extract variable j, copy X to E, apply floor halving $j - 1$ times, and use the two-step annihilator on E. Sequentially, the annihilator maps any nonnegative integer z to $z - 2\lfloor z/2 \rfloor$, its low bit. NOT is $1 - x$, AND is $\lfloor (x + y)/2 \rfloor$, and OR is expressed by De Morgan. Starting $P = 1$, the sequential recurrence therefore evaluates every clause and leaves the CNF value in P.

The lowered recurrence need not agree during these extraction chunks; the construction does not rely on such agreement, because every earlier coordinate is assigned the full word-span tolerance. In the final two-step chunk, the annihilator is applied to P. Its stored summary has an all-zero P output row, so the lowered final P is zero for any incoming lowered carry, while the reference final P is the CNF value. Giving final P tolerance zero makes the instance safe exactly when no packed assignment satisfies the CNF. Packed and halved values stay in $[0, 2^m - 1]$, Boolean registers stay in $\{0, 1\}$, and the selected word contains all values; the construction size is polynomial. □

Corollary 7.4 (Conditional compactness obstruction)**.** *If every safe instance in either coNP-complete family had a certificate of polynomial size checkable in polynomial time, then* NP = coNP.

Proof. Such certificates would place a coNP-complete safety language in NP. Polynomial many-one reductions would then give coNP ⊆ NP; taking complements gives NP ⊆ coNP. □

8 What the executable evidence establishes

The test suite independently instantiates both arithmetic paths on all development and frozen prefixes, exhaustively compares the scalar dynamic program with enumeration under all three association rules and both rounding modes on an eight-step case, checks every stored certificate, rejects parser and semantic mutations, and validates block projection and witness lifting. For the reductions it exhausts 832 circuit-assignment/mode combinations over all one-gate three-input and two-gate two-input shapes in the stated families, plus 867 CNF formulas and 6,474 packed assignments over one to three variables, including empty, duplicate-literal, and tautological clauses. These checks support implementation conformance for those finite domains. They do not turn Theorems 3.3–7.4 into machine-checked general proofs.